Teste Técnico

Next.js | NestJS | TypeScript | PostgreSQL | OpenAI API

CONTEXTO DO DESAFIO

Você vai construir uma pequena aplicação de gestão de compras (procurement). O sistema permite cadastrar fornecedores, registrar solicitações de compra e usar uma camada simples de IA para gerar automaticamente um parecer sobre cada solicitação, com prioridade e um breve resumo. O escopo é intencionalmente enxuto. Backend e frontend devem ser implementados em TypeScript.

PRAZO DE ENTREGA

Não há um prazo fixo definido. Você deve propor o prazo que considerar adequado para entregar o teste com qualidade e informar essa data (ou o número de dias) junto com o início do desafio. Essa proposta também é avaliada: considere sua disponibilidade real e o escopo do que está sendo pedido.

PARTE 1. BACKEND

1. Modelagem de dados

Crie um banco de dados PostgreSQL com as seguintes tabelas:

- suppliers: id, name, email, phone, category, created_at
- purchase_requests: id, item_name, quantity, supplier_id (FK, opcional), requester_name, justification, status, created_at
- ai_reviews: id, purchase_request_id (FK), priority, summary_text, generated_at

Validações simples: os campos item_name, quantity e requester_name são obrigatórios, quantity deve ser maior que zero e o e-mail do fornecedor deve ter formato válido.

2. API REST

Implemente os endpoints abaixo usando NestJS, em TypeScript:

- POST /suppliers: cria um fornecedor
- GET /suppliers: lista fornecedores
- POST /purchase-requests: cria uma solicitação de compra
- GET /purchase-requests: lista solicitações, com filtro por status
- GET /purchase-requests/:id: detalhe da solicitação, incluindo o parecer de IA (se existir)
- POST /purchase-requests/:id/review: chama a IA e gera o parecer automático
- PATCH /purchase-requests/:id/status: atualiza o status (pendente, aprovado, rejeitado)

3. Integração com IA

O endpoint POST /purchase-requests/:id/review deve:

- Buscar os dados da solicitação (item, quantidade, justificativa)
- Enviar um prompt para o modelo gpt-4o-mini pedindo uma classificação de prioridade (alta, média ou baixa) e um resumo de até 2 frases explicando o motivo
- Persistir o resultado em ai_reviews
- Retornar o parecer na resposta da API

A chave da API OpenAI será enviada por e-mail junto com este teste. Configure-a como variável de ambiente e não a inclua no repositório.

PARTE 2. FRONTEND (NEXT.JS)

4. Listagem de solicitações

Página principal com a lista de solicitações de compra. Deve incluir:

- Filtro por status
- Indicador visual de prioridade quando já existir parecer de IA

5. Detalhe e ação de IA

Ao clicar em uma solicitação, abrir uma página ou modal com:

- Dados da solicitação e do fornecedor, quando houver
- Parecer de IA, se existir (prioridade e resumo)
- Botão “Gerar parecer com IA”: chama o endpoint e exibe o resultado sem recarregar a página
- Opção de alterar o status da solicitação

6. Cadastro simples

Formulário simples para cadastrar um novo fornecedor e uma nova solicitação de compra.

ENTREGA

Repositório público no GitHub com:

- docker-compose.yml (opcional, mas bem-vindo)
- .env.example com todas as variáveis necessárias
- README.md com os passos para rodar localmente
- Migrations ou script SQL de criação do banco

Diferencial (opcional): testes unitários no backend, tratamento de erro da OpenAI (rate limit, timeout) e paginação na listagem.